NOCQ: A Constraint-Based Toolchain for Parity Games with Quantitative Conditions

Gonzalo Hernandez[1,3], Julian Gutierrez[2], and Guido Tack[1]

[1] Monash University, Australia {gonzalo.hernandez,guido.tack}@monash.edu
[2] University of Sussex, United Kingdom {J.Gutierrez}@sussex.ac.uk
[3] Universidad de Nariño, Colombia {gonzalo.hernandez}@udenar.edu.co

Abstract. Solving parity games is a cornerstone problem in formal verification and synthesis, with significant applications across industrial-scale system design. We present NOCQ, an efficient constraint-based toolchain designed to solve parity games under quantitative conditions. By leveraging the inherent duality of zero-sum game players, NOCQ implements several constraint programming (CP) techniques that exploit search space symmetries. Our experimental evaluation demonstrates that NOCQ performs competitively against state-of-the-art specialized solvers, often matching or exceeding their performance, while maintaining the modularity and flexibility of a CP-based framework. This flexibility allows users to easily incorporate additional constraints to search for preferred solutions, such as average quantitative performance requirements. We describe the tool's architecture, provide a comprehensive performance analysis, and discuss its extensibility. NOCQ is open-source and available at: https://github.com/gonzalohernandez/nocq.

Keywords: Constraint programming · Games on graphs · Verification.

1 Introduction

Parity games are two-player zero-sum infinite-duration games played on a labelled directed graph, where vertices represent states of the players in the game and edges represent their possible available actions at every state. Solving these games is central to several techniques in automated formal verification, in particular, when solving model checking and automated synthesis problems.

Since parity games are zero-sum, perfect-information games, solving them amounts to finding which player (usually called Player Even, or ○, and Player Odd, or □) has a winning strategy, from a given starting vertex in the graph. It is known that deciding the winner in parity games is an NP ∩ co-NP problem [21], but it is not known whether or not they can be solved in polynomial time; a problem that has remained open for nearly 50 years. Due to the importance of parity games in formal verification, several algorithms have been developed [1,5,20,22,30], all of them suffering from exponential running time limitations for specific classes of games. One such approach involves reducing the solution of a parity game to the solution of a SAT instance representing the game. The generated SAT instance is satisfiable if and only if Player Even has a winning strategy

in the original game. We further developed this idea in [19], introducing a novel CP-based algorithm which performs several orders of magnitude better than any other previous CP-based approach, and very competitively against the state-of-the-art algorithms/implementations for parity games in the literature [14,5,1] – often matching or exceeding their performance[4].

In this paper, we present NOCQ, a platform that implements the main algorithms developed in [19] for parity games and extended with new functionalities that allows one to include additional *quantitative* restrictions, including *energy* [9] and *mean-payoff* [10] constraints, which are useful when reasoning about additional long-term average performance – everything within the same SAT/CP-based framework. This tool paper briefly presents the problem it solves and the underlying algorithms that NOCQ is built upon, but focuses on the tool itself (its architecture, implementation, and usage) and some experimental results that demonstrate its performance in practice, in particular when applied to games with additional quantitative (energy and/or mean-payoff) constraints.

2 Preliminaries and Overview

This section presents the graph games that NOCQ solves, and some translations into Boolean Satisfiability (SAT) and Satisfiability Modulo Theories (SMT).

Arenas, Strategies, and Plays. A game arena is a tuple $\mathcal{A} = (V_\bigcirc, V_\square, E, \Omega, Q)$, where $V = V_\bigcirc \cup V_\square$ is a finite set of vertices partitioned between Player $\bigcirc$ (0) and Player $\square$ (1), $E \subseteq V \times V$ is a set of directed edges where every vertex has at least one successor, $\Omega : V \to \{0, \ldots, d\}$ is a priority mapping, and $Q : E \to \mathbb{Z}$ is a weight function. A *positional strategy* for player $i \in \{\bigcirc, \square\}$ is a function $f_i : V_i \to V$ such that $(v, f_i(v)) \in E$. A pair of strategies $(f_\bigcirc, f_\square)$ uniquely determines an infinite path $\pi = v_0 v_1 \ldots$, called a *play*. We denote by $\mathrm{Inf}(\pi)$ the set of priorities $\{\Omega(v) \mid v \text{ occurs infinitely often in } \pi\}$.

Winning Conditions. A play π is evaluated against three different criteria:

1. **Parity (ω):** Satisfied if $\max(\mathrm{Inf}(\pi))$ is even.
2. **Energy (η):** For $E_{init} \in \mathbb{N}$, satisfied if $\forall n \geq 0 : E_{init} + \sum_{i=0}^{n-1} Q(v_i, v_{i+1}) \geq 0$.
3. **Mean-Payoff (μ):** Satisfied if $\liminf_{n\to\infty} \frac{1}{n} \sum_{i=0}^{n-1} Q(v_i, v_{i+1}) \geq 0$.

We define Player $\bigcirc$'s winning goal as the combined condition formed by the conjunction of these requirements: $\gamma_\bigcirc = \omega \wedge \eta \wedge \mu$. As these are zero-sum games, Player $\square$ wins if the play satisfies $\gamma_\square = \neg\gamma_\bigcirc$. Given an initial vertex v_0, a strategy is said to be winning from v_0 for Player $\bigcirc$ (Player $\square$) if it only generates winning plays for Player $\bigcirc$ (Player $\square$) starting at v_0, for all strategies of the other player. It is known that for every vertex $v \in V$ in these games, one of the two players always has a winning strategy from that vertex. Solving a zero-sum game is deciding, for every $v \in V$, which player has a winning strategy from that vertex.

[4] Paper [19] is currently under review with notification due on 9 February. Preliminary reviews indicated strong support (two accept and one best paper nomination).

SMT/SAT Encodings. Beyond dedicated algorithms, parity games can be solved by encoding them into constraint-based formalisms like SAT or SMT [18]. This approach reduces the existence of a winning strategy for player ○ to the satisfiability of a formula Φ. We define Boolean variables S_v for $v \in V$ and $T_{v,w}$ for $(v,w) \in E$, representing whether vertices and edges belong to a winning strategy for Player ○. The structural winning conditions are:

- $\Phi_\exists = \bigwedge_{v \in V_\bigcirc} (S_v \to \bigvee_{w \in vE} T_{(v,w)})$
- $\Phi_\forall = \bigwedge_{v \in V_\square} (S_v \to \bigwedge_{w \in vE} T_{(v,w)})$
- $\Phi_V = \bigwedge_{w \in V, w \neq v_0} \left((\bigvee_{v \in Ev} T_{(v,w)}) \to S_w \right)$

such that, $\Phi_\exists$ and $\Phi_\forall$ ensure valid moves for Player ○ and □, respectively, while Φ_V ensures consistency between edge and vertex selection. To satisfy the parity condition ω, we employ a *discrete progress measure* [20] to rule out cycles where the maximum priority is odd. We introduce integer variables x_p^v for each $v \in V$ and each odd priority $p \in \mathrm{Odd}(G)$, where $\mathrm{Odd}^{>k}(G) = \{p \in \mathrm{Odd}(G) \mid p > k\}$. The progress constraints are $\Phi_A = \bigwedge_{(v,w) \in E} (T_{v,w} \to \Psi_{v,w})$, where:

$$\Psi_{v,w} = \begin{cases} \bigwedge_{p \in \mathrm{Odd}^{>\Omega(w)}} (x_p^v \geq x_p^w) & \text{if } \Omega(w) \text{ is even} \\ (x_{\Omega(w)}^v > x_{\Omega(w)}^w) \wedge \bigwedge_{p \in \mathrm{Odd}^{>\Omega(w)}} (x_p^v \geq x_p^w) & \text{if } \Omega(w) \text{ is odd.} \end{cases}$$

These constraints can be solved using SMT technologies: Satisfiability of $S_{v_0} \wedge \Phi_\exists \wedge \Phi_\forall \wedge \Phi_V \wedge \Phi_A$ determines if Player ○ wins from v_0. Alternatively, Φ_A can be bit-blasted into pure CNF for use with modern SAT solvers [18].

Solving Quantitative Parity Games. We solve quantitative parity games by encoding them into a Constraint Satisfaction Problem (CSP) with a specialized propagator [19], as follows:

Definition 1. *The CSP for an arena $G = (V, E, \Omega, Q)$ starting at v_0 from the perspective of player p is $\mathcal{P} = (\mathcal{X}, \mathcal{D}, \mathcal{C})$, where $\mathcal{X} = \{\mathcal{V}_v\}_{v \in V} \cup \{\mathcal{E}_e\}_{e \in E}$ are Boolean variables, and the constraints $\mathcal{C}$ are defined in the following way:*

$\mathcal{C}_{v_0}$: $\quad \mathcal{V}_{v_0} = \top$

$\mathcal{C}_\mathsf{p}$: $\quad \bigwedge_{v \in V_\mathsf{p}} (\mathcal{V}_v \to (\sum_{w \in vE} \mathcal{E}_{(v,w)} = 1))$

$\mathcal{C}_{\overline{\mathsf{p}}}$: $\quad \bigwedge_{v \in V_{\overline{\mathsf{p}}}} (\mathcal{V}_v \to \bigwedge_{w \in vE} \mathcal{E}_{(v,w)})$

$\mathcal{C}_E$: $\quad \bigwedge_{w \in V, w \neq v_0} ((\bigvee_{v \in Ew} \mathcal{E}_{(v,w)}) \to \mathcal{V}_w)$

$\mathcal{C}_{NOC}(f)$: $\quad \bigwedge_{v_0, \ldots, v_k} \left(\left(\bigwedge_{i=0}^{k-1} \mathcal{E}_{(v_i, v_{i+1})} \right) \to \bigwedge_{j=0}^{k-1} ((v_j = v_k) \to f(v_j, \ldots, v_k)) \right)$

- $f_\omega(v_j, \ldots, v_k) = \max\{\Omega(v_i) \mid i \in j \ldots k\} \bmod 2 \neq \overline{\mathsf{p}}$
- $f_\eta(v_j, \ldots, v_k) = \sum_{i=j}^{k-1} Q(v_i, v_{i+1}) \geq 0$
- $f_\mu(v_j, \ldots, v_k) = \frac{1}{k-j} \sum_{i=j}^{k-1} Q(v_i, v_{i+1}) \geq \nu$

While $\mathcal{C}_{v_0}, \mathcal{C}_{\overline{\mathsf{p}}}$, and $\mathcal{C}_E$ enforce connectivity, $\mathcal{C}_{\mathsf{p}}$ ensures that p selects exactly one strategy edge per vertex. Moreover, $\mathcal{C}_{NOC}$ ensures that every reachable cycle is winning for p by satisfying the given combination of parity (ω), energy (η), and mean-payoff (μ) requirements. Lastly, in f_μ, parameter $\nu \in \mathbb{Z}$ is a given performance threshold used to evaluate the mean-payoff condition. This generalized construction allows NOCQ to search from the perspective of either player by swapping p and $\overline{\mathsf{p}}$. Satisfiability under p proves a winning strategy for p, while unsatisfiability (if the solver explores the full space) proves the win for $\overline{\mathsf{p}}$.

A No-Opponent-Cycle filter. We have developed a complete propagator based on Depth-first Search (DFS) to explore the arena. The resulting DFS tree captures all possible paths from v_0, and can be used to detect all existing cycles. We define the propagator in the context of a *Lazy Clause Generation* solver [25], to take advantage of the strong performance of this type of solver, in particular for problems that contain a very large number of Boolean clauses. Algorithm 1 presents this version of the propagator, which checks for cycles and reports failure if a cycle satisfying the winning condition for $\overline{\mathsf{p}}$ is detected.

Algorithm 1 ***NOCFilter***

Input: $path_V$, $path_E$, v, e, $\mathcal{E}$, *defined*
Output: $\top$ is valid; $\bot$ otherwise

```
 1: if v ∈ path_V then
 2:     i_v ← index(v, path_V)
 3:     m ← max{f(v') | v' ∈ path_V[i_v :]}
 4:     if ¬γ_p then
 5:         reason ← clausify(¬path_E[i_v :])
 6:         if not (E[e] ← (⊥, reason)) then return ⊥
 7: else if defined then
 8:     for e' ∈ edges(v) where E[e'] ≠ ⊥ do
 9:         p_V ← path_V ∪ {v}
10:         p_E ← path_E ∪ {e'}
11:         w ← target(e')
12:         def ← E[e'] = ⊤
13:         NOCEager( p_V, p_E, w, e', E, def )
14: return ⊤
```

Algorithm 1 implements the No-Opponent-Cycle propagator using a recursive DFS traversal. It maintains the current exploration state via $path_V$ and $path_E$. Parameter $\mathcal{E}$ tracks the Boolean decision variables for arena edges. The core logic occurs when a cycle is detected (Line 1). The propagator evaluates γ_{p}—the specific combination of parity (ω), energy (η), and mean-payoff (μ) objectives in the game. If the cycle violates the winning criteria for p, the propagator generates a *conflict clause* consisting of the negation of the edges currently in $path_E$ (Line 5). This clause effectively *learns* the failing partial assignment, allowing the underlying SAT/CP solver to perform non-chronological backtracking and prune all other search branches that can lead to the same non-winning cycle.

3 Tool Description

In this section, we describe the implementation of NOCQ, a C++ framework developed to model and solve parity games with quantitative conditions. Unlike *Oink* [14], which focuses on explicit-state systems and symbolic algorithms, NOCQ is designed to leverage Constraint Programming (CP) and SAT technologies. The source code, implementation details, usage instructions, and a number of examples are available at `https://github.com/gonzalohernandez/nocq`.

Core Architecture and Modules

NOCQ is organised into four independent modules: *Game Manager*, *CPSModels*, *SATEncoding*, and *Utils*. This modular design ensures flexibility, allowing the tool to be used either as a standalone CP/SAT solver or as a reusable library.

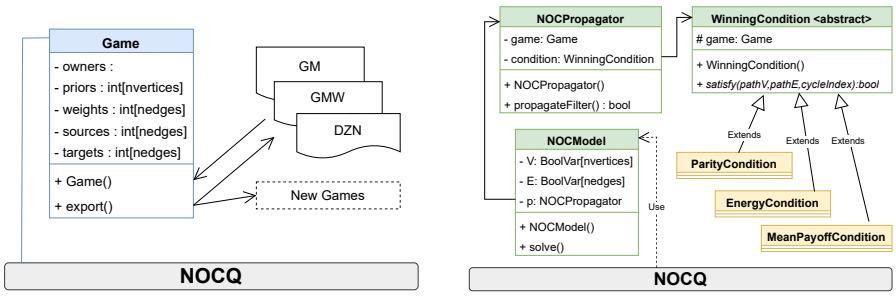
Game Manager. As shown in Figure 1a, the `Game` class serves as the core data structure of NOCQ. It captures essential game attributes, including vertex ownership (`owners`), priorities (`priors`), and edge weights (`weights`). In addition, it maintains the arena topology through `sources` and `targets` arrays for all edges.

Input and Procedural Generation. The framework supports standard formats used to represent games, such as GM [26], PG [14], and DZN [28]. We also define the GMW (`.gmw`) format, an extension of GM specifically designed to store edge weights. Moreover, following Parys [26], `Game` supports the procedural generation of *Random*, *Jurdziński*, and *MLadder* games for benchmarking purposes.

Usage Example. Users can interact with the Game Manager either to load existing benchmarks or to generate new game instances. The first command below loads a parity game and assigns random edge weights in the range $[-20, 20]$; by default, the solver maximizes for the parity reward. The second line generates a Jurdziński game with 15,000 levels and 50 blocks, set to minimize the reward:

```
$ ./nocq --gm /pgsolver/randomgame-20000-10-1-20-0.gm --weights -20 20
$ ./nocq --jurd 15000 50 --weights -50 30 --min
```

Constraint-Based Solving (CPSModels). Figure 1b illustrates how NOCQ implements the CSP model in Definition 1 and integrates the No-Opponent-Cycle propagators described in Section 2. A key design principle of NOCQ is model independence: the underlying model can be interfaced with different constraint programming (CP) engines. Once a game is loaded or created, an instance of `NOCModel` is instantiated and used to solve the problem. The `NOCModel` searches for solutions by enforcing all constraints, including C_{NOC} from Definition 1. This constraint is implemented through `NOCPropagator`, which uses CP techniques to evaluate winning conditions over all relevant cycles in the input game.



(a) NOCQ Game Manager. (b) NOCQ constraint-based solver.

Fig. 1: Overview of the main NOCQ Toolkit Components.

In the current version, three different winning conditions are implemented. The system is extensible to incorporate new winning conditions by extending the `WinningCondition` class with additional objectives. The framework has been validated using two state-of-the-art solvers:

- **Chuffed** [13], a lazy clause generation solver.
- **Gecode** [27], a comprehensive constraint-based toolkit.

Usage Example. Once a game has been loaded or created, one can invoke the constraint-based solver from the command line. Several parameters are available to tailor the solving process. The following command solves a mean-payoff parity game from a given initial vertex (876), and reports the execution times:

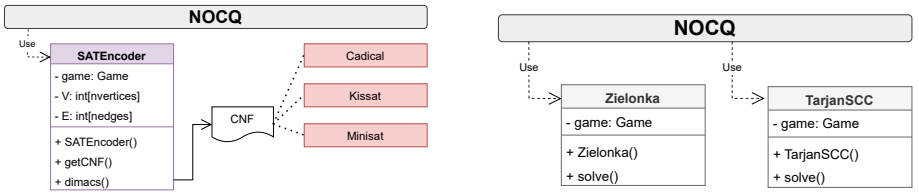
```
$ ./nocq --gm /pgsolver/randomgame-50000-10-1-20-0.gm --noc
    --init 876 --print-times --parity
    --mean-payoff --weights -50 50 --threshold 20
```

The tool then produces output in the following format (time in msecs), indicating that Player □ (odd) has a winning strategy from vertex 876:

```
Game creation time : 0.58195
Init time : 0.456748
Solving time : 0.0396906
Mem used : 210.043
876: ODD
```

The architecture of NOCQ is very modular, allowing new CP solvers to be integrated into the whole toolchain using the model developed in [19].

SAT Encoding. NOCQ also provides a dedicated SAT encoding pipeline, enabling parity games to be dispatched to a number of high-performance SAT solvers, including *CaDiCaL* [6], *Kissat* [7], and *MiniSat* [16]. Figure 2a illustrates the `SATEncoder` class, which implements the encoding techniques proposed by Heljanko *et al* [18], where a reduction to SAT is used, as described in Section 2.



(a) NOCQ SAT encoding. (b) Auxiliary algorithms in NOCQ.

Fig. 2: Overview of additional components of NOCQ.

The `SATEncoder` maintains references to the `Game` instance and maps graph components to integer representations of vertices (V) and edges (E). The module produces standard CNF output in DIMACS format, ensuring compatibility with any compliant SAT solver. We note that the SAT reduction proposed in [18] is restricted to parity objectives/games only, and does not support quantitative winning conditions such as energy or mean-payoff, as we do within NOCQ.

Usage Example. The following example encodes the `towersofhanoi-5` game from the pgsolver collection [17], starting from vertex 15, into CNF format. The encoding targets reward minimization, as described in [18]. The resulting output, a DIMACS file, can then be passed onto any SAT solver, such as CaDiCaL:

```
$ ./nocq --gm /pgsolver/towersofhanoi-5.gm --sat-encoding tower5.cnf --
    init 15
$ ./cadical tower5.cnf
```

Auxiliary Algorithms (Utils). Beyond CP and SAT technologies, NOCQ includes procedures for evaluation and preprocessing, as illustrated in Figure 2b:

- **Zielonka** [2]: parity games algorithm with improved attractor computation.
- **TarjanSCC** [29]: efficient SCC decomposition of game arenas.

Usage Example. The command below first generates a random game with 20 vertices, 10 priorities, and outgoing degree between 1 and 5, then solves it using the Zielonka Recursive Algorithm (ZRA), and finally exports the arena:

```
$ ./nocq --rand 20 10 1 5 --zra --print-solution --export-gm rand.gm
```

Below is NOCQ's output, indicating the vertices from which each player has a winning strategy, that is, the *winning regions* for each player in the game:

```
EVEN {12,4,7}
ODD {11,14,15,2,3,1,6,17,16,5,13,18,0,19,9,10,8}
```

The command below performs SCC decomposition on the exported game:

```
$ ./nocq --gm rand.gm --scc
```

Output:

```
{11}
{19,15,2,3,13,9,10,7,18,17,5,12,6,14,4,16,0}
{1}
{8}
Total SCCs: 4
```

NOCQ users can view the full list of supported parameters using `./nocq -help`. A more in-depth explanation of specific configurations and advanced usage is available as part of the documentation in the Git repository of NOCQ.

Exploiting Problem Duality through Parallel Solving. The performance of game solvers is strongly influenced by the topological and quantitative structure of the arena. A key observation is the *asymmetry in computational cost*: game instances that are difficult to solve from the perspective of player p are often easier (in practical terms) from the perspective of $\overline{\mathsf{p}}$, and vice versa. To exploit this duality, NOCQ executes the CSP concurrently from both perspectives in separate threads. The computation terminates as soon as one thread identifies a winning strategy or proves its absence. This parallel approach mitigates worst-case behavior by leveraging the more computationally favorable viewpoint, which depends on the particular game that is given to NOCQ as input.

4 Experiments and Evaluation

In this section, we evaluate NOCQ from two primary perspectives. First, we assess the performance of the tool when solving games with parity conditions only. We compare NOCQ against state-of-the-art algorithms using standard benchmarks.

For the SAT backends, we use the following solvers: **CaDiCaL** (v2.1.3) [6], **MiniSat** (v2.2.0) [16], **Kissat** (v4.0.2) [7], **zChaff** (v2007.3.12) [24]. To ensure state-of-the-art comparisons, we incorporated **Oink** (latest GitHub version) [14], which includes high-performance algorithms such as Priority Promotion (PP and PP+) and Parys' improvements (PAR). We also included Zielonka's Recursive Algorithm (ZRA), with an improved attractor strategy as presented in [1].

We then evaluated NOCQ's performance when solving parity games with quantitative conditions. The objective was to determine the variance in execution time when NOCQ evaluates different combinations of $\gamma_p = \omega \otimes \eta \otimes \mu$, where $\otimes$ denotes the product composition of the parity, energy, and mean-payoff goals. The benchmarks used were taken from the PGSolver Library [17] and Keiren's benchmarks for parity games [23]. The games were classified into 4 categories:

- **Equivalence Checking (EqC):** 77 instances from formal verification tasks (*e.g.*, strong and weak bisimulation), featuring complex structural connectivity typical of system architectures.
- **Model Checking (MoC):** 106 organic benchmarks derived from μ-calculus model checking, representing practical real-world verification challenges.

– **PGSolver Collection (PGS):** 161 games taken from difficult families (*e.g.*, Jurdziński, ladder, and clique games [3]) used to evaluate performance against theoretical worst-case bounds of several decision procedures.
– **Random Games (RaG):** 125 game instances with varied priority distributions, serving as a baseline for general robustness and average-case efficiency.

Moreover, these benchmarks were partitioned further into two categories, small-scale and large-scale, depending on the size of the input games. This distinction allows us to measure the feasibility of SAT-reduction methods on smaller instances while testing the scalability of NOCQ against dedicated solvers on larger game graphs. All experiments were conducted on an Intel Xeon 8260 CPU with 24 cores (non-hyperthreaded) and 268.55 GB of RAM running Linux.

Table 1: Performance (in time) of Algorithms on small-scale instances (Max vertices: EqC/MoC $\approx 10^6$, PGS/RaG $\approx 10^3$) and large-scale instances (Max vertices: ≈ 35 million). Average time measured in seconds using PAR2 Scoring.

Algorithm	**EquivalenceCh**		**ModelCh**		**PGSolver**		**RandomG**	
	Small	Large	Small	Large	Small	Large	Small	Large
Oink(PP)	0.663	24.775	0.327	9.547	**0.126**	44.949	0.001	0.016
Oink(PP+)	0.649	24.876	0.324	9.557	0.126	44.922	0.001	0.015
Oink(PAR)	0.544	24.159	0.294	8.718	2.580	182.643	**0.001**	**0.015**
ZRA(ImpAttr)	**0.190**	**14.490**	**0.172**	**3.929**	7.237	201.804	0.002	0.053
NOCQ(Ours)	0.399	16.169	0.289	8.838	5.043	**32.189**	0.003	0.078
SAT(CaDiCaL)	16.114	—	15.524	—	292.021	—	1.100	—
SAT(Kissat)	76.424	—	28.924	—	294.794	—	1.113	—
SAT(MiniSat)	32.594	—	22.443	—	306.186	—	1.384	—

Table 1 presents a consolidated performance comparison. On small-scale games, NOCQ outperforms SAT-based methods by up to three orders of magnitude and remains competitive with non-CP algorithms, typically within a factor of two of the fastest solvers. For large-scale instances, SAT-based approaches are omitted due to their inability to scale within reasonable resource limits. In this context, NOCQ demonstrates significant robustness: while ZRA [1] remains the fastest for structured benchmarks (EqC and MoC), NOCQ notably outperforms all tested Oink variants and ZRA in the high-complexity PGSolver collection.

Quantitative Goals and Verification. One of NOCQ's main features is the ability to handle quantitative goals within the same CP-based framework. NOCQ implements a core algorithm designed to compute positional strategies, which are optimal in case only qualitative (parity) or only quantitative goals are given. However, when combined, positional strategies may no longer be optimal [12,8]. As such, the solutions that NOCQ computes are for games where only positional strategies are allowed. In this context, the next set of benchmarks demonstrates NOCQ's performance when solving such kinds of games when enriched with additional quantitative (energy and/or mean-payoff) requirements.

Table 2: Performance (in time) of NOCQ with different winning conditions on small-scale and large-scale instances. Same configuration as in Table 1.

Conditions	**EquivalenceCh**		**ModelCh**		**PGSolver**		**RandomG**	
γ	Small	Large	Small	Large	Small	Large	Small	Large
ω	0.001	2.308	0.001	3.750	2.815	15.095	0.001	0.082
$\omega \otimes \eta$	0.001	2.308	3.158	3.750	2.505	12.119	0.001	0.063
$\omega \otimes \mu$	0.001	2.308	0.001	3.750	8.850	17.250	3.601	6.050
$\omega \otimes \eta \otimes \mu$	0.001	2.308	1.053	3.750	3.756	15.241	4.316	7.010
η	0.001	2.308	1.439	3.750	1.686	12.010	0.001	0.065
μ	0.001	4.616	1.053	3.750	3.771	15.067	4.202	10.697
$\eta \otimes \mu$	0.001	4.616	1.374	3.750	0.850	14.833	1.861	4.355

Our analysis of the winning conditions in Table 2 reveals that the performance is significantly influenced by the specific combination of objectives. While basic parity (ω) and energy (η) conditions demonstrate the highest efficiency, the introduction of mean-payoff requirements (μ) acts as the predominant factor in execution time, nearly tripling solving time in some RandomGames instances. Interestingly, the data suggests a high level of compositional efficiency within the NOCQ framework: the transition from a single μ condition to a combined $\omega \otimes \eta \otimes \mu$ objective results in only a marginal performance penalty. This indicates that once the solver accounts for the most restrictive quantitative constraint (mean-payoff in this case), the addition of further qualitative (parity) or quantitative (energy-based) conditions via the $\otimes$ operator does not lead to a cumulative or exponential increase in solving time, evidence of the robustness of the underlying constraint-based architecture upon which NOCQ is built.

To evaluate these conditions, we enriched the existing benchmarks with random edge weights in the $[-50, 50]$ interval using a uniform distribution.

5 Conclusions and Related Work

As demonstrated by our experimental evaluation, NOCQ is an efficient toolchain that leverages a constraint-based approach to solve games on graphs with performance competitive with state-of-the-art algorithms used in formal verification to solve parity games. A key advantage of NOCQ lies in its ability to solve parity games combined with energy and mean-payoff conditions simultaneously, maintaining a high performance without increasing implementation complexity. Because NOCQ is built upon CP, the model is inherently extensible; new constraints can be added seamlessly without modifying the underlying solver architecture.

Finally, several tools have been developed for quantitative games only – *e.g.*, see [4] and references therein. But, much less work has been done regarding practical tools for parity games (or ω-regular games more generally) with additional quantitative requirements. In [11,15] two attempts were made, but in such cases full parity games with quantitative objectives were not evaluated in practice; Our results provide a step forward where the case for positional strategies is accounted for in complex parity games with additional quantitative requirements.

References

1. Aggarwal, S., de la Banda, A.S., Yang, L., Gutierrez, J.: A matrix-based approach to parity games. In: Sankaranarayanan, S., Sharygina, N. (eds.) Tools and Algorithms for the Construction and Analysis of Systems. pp. 666–683. Springer Nature Switzerland, Cham (2023)
2. Araya, I., Neveu, B., Trombettoni, G.: Exploiting common subexpressions in numerical csps. pp. 342–357 (09 2008). https://doi.org/10.1007/978-3-540-85958-1_23
3. Benerecetti, M., Dell'Erba, D., Mogavero, F.: Solving parity games via priority promotion. In: Chaudhuri, S., Farzan, A. (eds.) Computer Aided Verification. pp. 270–290. Springer International Publishing, Cham (2016)
4. Benerecetti, M., Dell'Erba, D., Mogavero, F.: Solving mean-payoff games via quasi dominions. Inf. Comput. **297**, 105151 (2024). https://doi.org/10.1016/J.IC.2024.105151, https://doi.org/10.1016/j.ic.2024.105151
5. Benerecetti, M., Dell'Erba, D., Mogavero, F., Schewe, S., Wojtczak, D.: Priority promotion with parysian flair. Journal of Computer and System Sciences **147**, 103580 (2025). https://doi.org/https://doi.org/10.1016/j.jcss.2024.103580, https://www.sciencedirect.com/science/article/pii/S0022000024000758
6. Biere, A., Faller, T., Fazekas, K., Fleury, M., Froleyks, N., Pollitt, F.: CaDiCaL 2.0. In: Gurfinkel, A., Ganesh, V. (eds.) Computer Aided Verification - 36th International Conference, CAV 2024, Montreal, QC, Canada, July 24-27, 2024, Proceedings, Part I. Lecture Notes in Computer Science, vol. 14681, pp. 133–152. Springer (2024). https://doi.org/10.1007/978-3-031-65627-9_7
7. Biere, A., Faller, T., Fazekas, K., Fleury, M., Froleyks, N., Pollitt, F.: CaDiCaL, Gimsatul, IsaSAT and Kissat entering the SAT Competition 2024. In: Heule, M., Iser, M., Järvisalo, M., Suda, M. (eds.) Proceedings of SAT Competition 2024: Solver, Benchmark and Proof Checker Descriptions. Department of Computer Science Report Series B, vol. B-2024-1, pp. 8–10. University of Helsinki, Finland (2024), https://cca.informatik.uni-freiburg.de/papers/BiereFallerFazekasFleuryFroleyksPollitt-SAT-Competition-2024-solvers.pdf
8. Chatterjee, K., Doyen, L.: Energy parity games. CoRR **abs/1001.5183** (2010), http://arxiv.org/abs/1001.5183
9. Chatterjee, K., Doyen, L.: Energy parity games. Theor. Comput. Sci. **458**, 49–60 (2012). https://doi.org/10.1016/J.TCS.2012.07.038, https://doi.org/10.1016/j.tcs.2012.07.038
10. Chatterjee, K., Henzinger, M., Svozil, A.: Faster algorithms for mean-payoff parity games. In: Larsen, K.G., Bodlaender, H.L., Raskin, J. (eds.) 42nd International Symposium on Mathematical Foundations of Computer Science, MFCS 2017, Aalborg, Denmark, August 21-25, 2017. LIPIcs, vol. 83, pp. 39:1–39:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2017). https://doi.org/10.4230/LIPICS.MFCS.2017.39, https://doi.org/10.4230/LIPIcs.MFCS.2017.39
11. Chatterjee, K., Henzinger, T.A., Jobstmann, B., Singh, R.: QUASY: quantitative synthesis tool. In: Abdulla, P.A., Leino, K.R.M. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 17th International Conference, TACAS 2011, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2011, Saarbrücken, Germany, March 26-April

3, 2011. Proceedings. Lecture Notes in Computer Science, vol. 6605, pp. 267–271. Springer (2011). https://doi.org/10.1007/978-3-642-19835-9_24, https://doi.org/10.1007/978-3-642-19835-9_24
12. Chatterjee, K., Henzinger, T.A., Jurdzinski, M.: Mean-payoff parity games. In: 20th IEEE Symposium on Logic in Computer Science (LICS 2005), 26-29 June 2005, Chicago, IL, USA, Proceedings. pp. 178–187. IEEE Computer Society (2005). https://doi.org/10.1109/LICS.2005.26, https://doi.org/10.1109/LICS.2005.26
13. Chu, G., Stuckey, P.J.: Chuffed: A lazy clause generation constraint solver. GitHub repository (2018), https://github.com/chuffed/chuffed
14. van Dijk, T.: Oink: An implementation and evaluation of modern parity game solvers. In: Beyer, D., Huisman, M. (eds.) Tools and Algorithms for the Construction and Analysis of Systems. pp. 291–308. Springer International Publishing, Cham (2018)
15. Dziadek, S., Fahrenberg, U., Schlehuber, P.: ω-regular energy problems. Formal Aspects Comput. **37**(1), 4:1–4:30 (2025). https://doi.org/10.1145/3678265, https://doi.org/10.1145/3678265
16. Eén, N., Sörensson, N.: An extensible SAT-solver. In: Giunchiglia, E., Tacchella, A. (eds.) Theory and Applications of Satisfiability Testing (SAT 2003). Lecture Notes in Computer Science, vol. 2919, pp. 502–518. Springer (2003). https://doi.org/10.1007/978-3-540-24605-3_37
17. Friedmann, O., Lange, M.: The PGSolver Collection of Parity Game Solvers, Version 4.1 (June 2017), https://github.com/tcsprojects/pgsolver/blob/master/doc/pgsolver.pdf, accessed: 2025-03-13
18. Heljanko, K., Keinänen, M., Lange, M., Niemelä, I.: Solving parity games by a reduction to SAT. Journal of Computer and System Sciences **78**(2), 430–440 (2012). https://doi.org/https://doi.org/10.1016/j.jcss.2011.05.004, https://www.sciencedirect.com/science/article/pii/S0022000011000535, games in Verification
19. Hernandez, G., Garcia, J., Gutierrez, J., Tack, G.: No-opponent-cycle propagators for solving parity games. In: Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR 2026) (2026), under review
20. Jurdziński, M.: Small progress measures for solving parity games. In: Reichel, H., Tison, S. (eds.) STACS 2000. pp. 290–301. Springer Berlin Heidelberg, Berlin, Heidelberg (2000)
21. Jurdziński, M.: Deciding the winner in parity games is in UP $\cap$ co-UP. Information processing letters **68**(3), 119–124 (1998)
22. Jurdziński, M., Morvan, R., Thejaswini, K.S.: Universal algorithms for parity games and nested fixpoints. In: Lecture notes in computer science, pp. 252–271. Lecture Notes in Computer Science, Springer Nature Switzerland, Cham (2022)
23. Keiren, J.J.A.: Benchmarks for parity games (extended version) (2015), https://arxiv.org/abs/1407.3121
24. Moskewicz, M.W., Madigan, C.F., Zhao, Y., Zhang, L., Malik, S.: Chaff: Engineering an efficient SAT solver. In: Proceedings of the 38th Annual Design Automation Conference (DAC 2001). pp. 530–535. ACM (2001). https://doi.org/10.1145/378239.379017
25. Ohrimenko, O., Stuckey, P.J., Codish, M.: Propagation via lazy clause generation. Constraints An Int. J. **14**(3), 357–391 (2009). https://doi.org/10.1007/S10601-008-9064-X, https://doi.org/10.1007/s10601-008-9064-x
26. Parys, P.: Parity games: Zielonka's algorithm in quasi-polynomial time. arXiv.org (2019)

27. Shulte, C., Tack, G., Lagerkvist, M.: Modeling and programming with gecode (2019), https://www.gecode.org/doc-latest/MPG.pdf
28. Stuckey, P.J., Kim, M., Guido, T.: The minizinc handbook (2020), https://www.minizinc.org/doc-latest/en/index.html
29. Tarjan, R.: Depth-first search and linear graph algorithms. SIAM journal on computing **1**(2), 146–160 (1972)
30. Zielonka, W.: Infinite games on finitely coloured graphs with applications to automata on infinite trees. Theoretical computer science **200**(1), 135–183 (1998)